

Sesión 101

Inteligencia Artificial con Python

Instituto Politécnico Nacional
Centro de Investigación en Computo

M. Alan Badillo Salas

alan@nomadacode.com

23 de mayo de 2026

Resumen

Bienvenido al curso de Inteligencia Artificial con Python, en esta clase revisaremos los fundamentos de Python de forma práctica y orientados a la IA. Revisaremos el funcionamiento de las principales librerías como Numpy, Matplotlib, Pandas y Seaborn. Finalmente aprenderemos a utilizar y configurar diversos entornos de desarrollo.

Introducción

Python es un lenguaje de programación versátil que nos permite ejecutar código de forma rápida y flexible. La gran disponibilidad de paquetes científicos y de inteligencia artificial disponibles, lo hacen atractivo para la ciencia de datos, el aprendizaje automático y la inteligencia artificial.

En estas notas de la sesión, revisaremos tres puntos principales de Python orientados al cómputo científico:

1. El flujo de escalares y vectores de datos
2. La visualización de datos
3. La carga masiva de datos

Posteriormente, realizaremos una serie de ejercicios para fortalecer la visión general de la IA en cinco problemas de interés:

1. Escalares y vectores aleatorios, con y sin semilla
2. La función objetivo y optimizador ingenuo
3. La función de pérdida y los parámetros desconocidos
4. El modelo de regresión lineal
5. La curva logística como función de activación y el perceptrón

Con esto, tendremos las herramientas y visión suficientes para comenzar a adentrarnos al mundo de la inteligencia artificial, las redes neuronales, el aprendizaje profundo y los transformadores.

1. El flujo de escalares y vectores de datos

En Python un dato simple se considera escalar y estos pueden ser de diferentes tipos como números (enteros o decimales), textos (cadenas o secuencias de caracteres entrecomillados) o valores lógicos (booleanos).

Al contrario, un vector de datos representa una secuencia de valores escalares dispuestos sobre un mismo objeto o colección. Estos pueden ser representados por tuplas (de python), listas (simples de python), arreglos (de numpy) o series (de pandas). El vector permite acceder a los valores indexados y en algunos casos se puede reemplazar el valor, extender el vector o realizar flujos de agregación (resumir los valores a escalares como la suma, media, etc.).

1.1. Ejemplo 1 - Calcular la diferencia de ventas

Para ejemplificar esto, pensemos en una sucesión de valores de ventas diarias, estas representan la cantidad de productos similares al día y permiten observar el flujo de ventas que ocurren día con día en un negocio.

Podríamos pensar en las ventas diarias como:

$$v_1, v_2, v_3, \dots, v_n \quad (1)$$

Entonces cada v_j representará el total de productos vendidos el día j . Un problema común es comparar las ventas actuales contra las previas, para determinar la diferencia entre las ventas del día actual y el día anterior, y así determinar una tendencia.

$$v_2 - v_1, v_3 - v_2, v_3 - v_2, \dots, v_n - v_{n-1} \quad (2)$$

Así, tendremos $n - 1$ diferencias que nos permitirán saber cuántas ventas ganamos o perdimos respecto al día anterior, este es un clásico cálculo en la serie *LAG*, que contiene los mismos valores pero desplazados una o más posiciones, esto nos permite comparar un valor actual contra un valor anterior o posterior (análisis de diferencias pasadas o futuras).

Una forma rápida de evaluar estos cálculos sin el uso de librerías, sería listando los valores de la venta, luego creando una copia desde el primero al último índice y finalmente desplazar la copia a la derecha para recorrer los valores anteriores y que coincidan en índice con los actuales.

Veamos un ejemplo de esto usando listas simples de python.

```
1  ventas = [13, 15, 17, 19, 32, 23, 22, 18, 21, 16, 12,
2      11]
3  print(ventas)
4
5  ventas_desplazadas = ventas[:]
6
7  ventas_desplazadas.insert(0, 0)
8  ventas_desplazadas.pop()
9
10 print(ventas_desplazadas)
11
12 ventas1 = ventas[1:]
13 ventas2 = ventas_desplazadas[1:]
14
15 print(ventas1)
16 print(ventas2)
17
18 diferencias = [v1 - v2 for v1, v2 in zip(ventas1,
19     ventas2)]
20 print(diferencias)
```

Sin embargo, no hay una única forma de resolver los problemas, ni tampoco de generar el código adecuado para conseguirlo. Por ejemplo, veamos cómo se haría en numpy.

```
1  import numpy
2
```

```

3  ventas = numpy.array([13, 15, 17, 19, 32, 23, 22, 18,
4      21, 16, 12, 11])
5  print(ventas)
6
7  ventas_desplazadas = numpy.roll(ventas, 1)
8
9  print(ventas_desplazadas)
10
11  ventas1 = ventas[1:]
12  ventas2 = ventas_desplazadas[1:]
13
14  print(ventas1)
15  print(ventas2)
16
17  diferencias = ventas1 - ventas2
18
19  print(diferencias)

```

Aunque al usar una librería, lo mejor sería pensar como esa librería hace las cosas, para no complicar innecesariamente el código:

```

1  import numpy
2
3  ventas = numpy.array([13, 15, 17, 19, 32, 23, 22, 18,
4      21, 16, 12, 11])
5  print(ventas)
6
7  diferencias = numpy.diff(ventas, 1)
8
9  print(diferencias)

```

2. La visualización de datos

Una potencia importante de Python consiste en poder generar gráficas fácilmente, y con esto se abre toda una rama del análisis de datos orientada a la visualización de datos.

La configuración de las gráficas requiere paciencia y muchas recetas memori-

zadas o consultadas para adaptar el título, el nombre de los ejes, la finidad y estilo de las series, puntos o barras. Pero, dejando de lado estas configuraciones finas, podemos pensar en formas directas de representar la información que queremos mostrar, algunos gráficos importantes son:

- **Histograma** - Muestra en barras la cantidad o frecuencia de datos que caen en un intervalo para un eje de datos, por ejemplo, si el eje de datos son las edades, el histograma determina en qué regiones se concentra la mayor cantidad de datos. Los intervalos o *bins* (cubetas) suelen ser finos para muchos datos o gruesos para pocos datos.
- **Barras** - Permiten el contraste entre un eje de datos categórico o discreto para ver la evolución de otro eje de datos usado como altura, generalmente, las barras se usan para observar cómo se distribuyen los datos en varias categorías (la altura sería el número de datos en la categoría o un peso que tiene cada categoría), aunque también es común ver una gráfica de barras mostrando valores que cambian según una serie discreta ordenada, por ejemplo, el salario por edad o la cantidad de ventas por día o fecha.
- **Líneas** - Las líneas permiten mostrar de forma más fina el desplazamiento ordenado de un eje de datos respecto a otro. Generalmente, el eje horizontal retendrá la información temporal o espacial de una sucesión ordenada de valores como fechas, índices o progresiones (como las edades), mientras que el eje vertical mantendrá un puntón de altura para cada valor horizontal, por ejemplo, el salario por edad, la diferencia de una venta con su anterior, etc.
- **Puntos** - Permite contrastar una nube de puntos para dos ejes de datos que no necesariamente estarán ordenados o son parcialmente ordenados, esto sirve para proponer modelos de curva que atraviese la nube de puntos intentando aproximar o inferir los valores esperados.
- **Densidad y contraste** - Se utiliza para entender la masa de concentración de un eje de datos, y además cuando se usa el contraste (*hue*), podemos ver si hay una separación de las densidades explicadas por algún eje discreto, por ejemplo, ver si la masa de edades se separa dado que conocemos si tiene empleo o no tiene empleo.
- **Mapa de calor** - Permite contrastar la concentración de valores que producen dos categorías cruzadas (dos ejes categóricos o discretos que se comparan elemento por elemento). Por ejemplo, se puede determinar si una persona en cierto grupo de edad tendrá empleo o no tendrá empleo,

usando el porcentaje de personas empleadas y no empleadas para cada grupo de edad.

- **Mapa geográfico** - Permite visualizar datos regionales más complicados como nube de coordenadas o polígonos regionales. Por ejemplo, una nube con las coordenadas de los hospitales, coloreadas por la demanda de cada hospital (incluso cada hospital podría representar un círculo cuyo radio es proporcional a su demanda). Otro ejemplo, podrían ser los atractores por municipio o colonia formando polígonos que indiquen factores interesantes como por qué candidato político votaría una región.

2.1. Ejemplo 2 - Visualizar el comportamiento de las ventas

Veamos como construir una gráfica que muestre las ventas y sus diferencias.

```
1 import numpy
2 from matplotlib import pyplot
3
4 ventas = numpy.array([13, 15, 17, 19, 32, 23, 22, 18,
5                       21, 16, 12, 11])
6
7 diferencias = numpy.diff(ventas, 1)
8
9 pyplot.plot(ventas)
10 pyplot.bar(numpy.arange(diferencias.size) + 1,
11             diferencias)
```

Y es importante recordar que siempre hay diferentes formas de hacer las cosas.

```
1 import pandas
2 from matplotlib import pyplot
3
4 ventas = pandas.Series([13, 15, 17, 19, 32, 23, 22, 18,
5                       21, 16, 12, 11])
6
7 diferencias = ventas.diff(1)
```

```

7
8 figure, axis = pyplot.subplots(2, 1, height_ratios=(1,
9     2))
10
11 diferencias.plot.bar(ax=axis[0])
12 ventas.plot(ax=axis[1])
13
14 pyplot.show()

```

3. La carga masiva de datos

Cuando necesitamos trabajar con datos de Excel, CSV, Bases de Datos o APIs, python comienza a diferenciarse de otros entornos y muestra su verdadero potencial.

Mediante pandas podemos cargar datos en forma de *DataFrames* que permitirán recuperar los datos de las filas y columnas extraídas como series de pandas que son en su base arreglos de numpy.

3.1. Ejemplo 4 - Analizar las ventas CSV

En el siguiente ejemplo se muestran las ventas cargadas desde el archivo CSV.

```

1 import pandas
2 from matplotlib import pyplot
3
4 ventas = pandas.read_csv("../..conjuntos/ventas.csv",
5     encoding="latin-1")
6
7 print(ventas)
8
9 ventas_conteos = ventas.groupby("fecha").size()
10
11 print(ventas_conteos)
12
13 diferencias = ventas_conteos.diff(1)
14
15 figure, axis = pyplot.subplots(2, 1, height_ratios=(1,
16     2))

```

```

16 |diferencias.plot.bar(ax=axis[0])
17 |ventas_conteos.plot(ax=axis[1])
18 |
19 |pyplot.show()

```

4. Escalares y vectores aleatorios, con y sin semilla

Los números aleatorios (pseudo-aleatorios) nos permiten crear procesos estocásticos simples para evitar generar puntos de partida de valores que desconocidos que quieran ser optimizados poco a poco, por ejemplo, los coeficientes de regresión.

Con numpy podemos crear un escalar aleatorio dentro de un intervalo uniforme, o mejor aún bajo una distribución normal, lo que permitiría tener un valor cercano a una media, pero con una desviación que permita alejarse bajo una probabilidad con la que sea cada vez difícil alejarse de dicho valor.

$$\beta \sim \mathbf{Normal}(\mu, \sigma) \quad (3)$$

Por ejemplo, si se desea un escalar, entonces haremos que $n = 1$, y si queremos un vector, entonces n será el tamaño del vector. Así, el valor será cercano a μ , con desviación estándar σ .

```

1 |beta = numpy.random.normal(mu, sigma, n)

```

5. La función objetivo y optimizador ingenuo

La función objetivo representa el modelo paramétrico que será la generalización de una función que no es específica si no se conocen sus parámetros, pero que al proponer algún parámetro fijado permitirá producir valores que intentarán comportarse como los datos reales (las respuestas observadas).

En general tenemos, una función que usa la información conocidas (las características x del problema) y las respuestas observada (el valor esperado y del problema). Esperando que la predicción del modelo se acerque a los valores observados.

$$y \approx \hat{y} = \phi(x, \beta) \quad (4)$$

Donde $\phi(x, \beta)$ representa el modelo paramétrico que depende de los parámetros inicialmente desconocidos β que deberán ser ajustados poco a poco, partiendo inicialmente de escalares o vectores aleatorios, según el número de variables desconocidas x . Es decir, si x es una sola característica, tendríamos generalmente β_1 y β_0 que modelarán fácilmente una regresión lineal simple, pero si tenemos $x_1, x_2, \dots, x_k$, necesitaremos $\beta_1, \beta_2, \dots, \beta_k$, además de β_0 que modelarán fácilmente una regresión lineal múltiple o una curva logística.

Por otro lado, para encontrar las β (parámetros desconocidos), podemos construir un optimizador ingenuo que vaya evaluando los mejores encontrados hasta el momento y los nuevos parámetros propuestos para medir los que produzcan menor error o menor pérdida.

$$\beta^* \sim \text{Normal}(0, 1) \quad (5)$$

Ahora, con los mejores parámetros conocidos (inicialmente aleatorios cercanos a cero), repetiremos para cada época en un número dado de épocas.

$$\beta^+ \sim \text{Normal}(\beta^*, 1) \quad (6)$$

Donde β^+ representan los parámetros cercanos a β^* que serán probados para ver si el modelo de predicción mejora.

Entonces, dentro de la misma época evaluamos $\hat{y}_*$ (respuesta producida por el modelo con los parámetros óptimos hasta el momento) y $\hat{y}_+$ (respuesta producida por el modelo con los parámetros de prueba cercanos a los parámetros óptimos, pero alejados aleatoriamente).

$$\begin{aligned} \hat{y}_* &= \phi(x, \beta^*) \\ \hat{y}_+ &= \phi(x, \beta^+) \end{aligned} \quad (7)$$

Finalmente, para determinar el error necesitaremos una función de pérdida $L(y, \hat{y})$ que permita medir la diferencia o error (la pérdida o acumulación del error) que se produce entre las respuestas reales observadas y las predicciones, generalmente el *MSE* (*Mean Square Error* / *Error Cuadrático Medio*) en un modelo de regresión o el *BCE* (*Binary Cross-Entropy* / *Entropía-Cruzada Binaria*) en una curva logística. Aunque podríamos también diseñar nuestra propia forma de medir el error entre las predicciones de respuesta ($\hat{y}$) y las respuestas reales observadas (y).

Así, dentro de la misma época obtenemos el error de predicción de $\hat{y}_*$ y $\hat{y}_+$ para concluir si β^+ es mejor que β^* (los parámetros de prueba son mejores

a los óptimos encontrados hasta el momento) y determinar actualizar los β^* si el error ε_+ es menor al error ε_+ .

$$\begin{aligned}\varepsilon_* &= \mathbf{L}(y, \hat{y}_*) \\ \varepsilon_+ &= \mathbf{L}(y, \hat{y}_+)\end{aligned}\tag{8}$$

Entonces, β^* se mantendrá igual mientras el error de pérdida ε_+ no mejore al error óptimo ε_* .

6. El modelo de regresión lineal

El modelo de regresión lineal, es un modelo simple que construye un predictor lineal $x \cdot \beta + \beta_0$ que forma una recta en el espacio para cada β (pendiente) y β_0 (interceptor o bias). Con lo que nuestra función objetivo para una sola característica x (*Modelo de Regresión Lineal Simple*) sería:

$$y \approx \hat{y} = \phi(x, \beta) = x \cdot \beta_1 + \beta_0\tag{9}$$

O para múltiples características $x_1, x_2, \dots, x_k$ (*Modelo de Regresión Lineal Múltiple*) sería:

$$y \approx \hat{y} = \phi(x, \beta) = \beta_0 + x_1 \cdot \beta_1 + x_2 \cdot \beta_2 + \dots + x_k \cdot \beta_k\tag{10}$$

Este modelo de regresión lineal, determina una recta para los valores de β fijados y pretende ajustarse lo mejor posible a la nube de puntos formados por la respuesta observada y .

Entre más cercanas estén las respuestas observadas a la recta, mejor predicción tendrá el modelo. Lo ideal es que las bandas (región que cubre de forma normal la predicción) atrape todos los puntos, sino el error entre lo observado y las predicciones no se comportará de forma normal y no podríamos garantizar que un punto fuera de las bandas tenga muy poca probabilidad de ocurrir (**Figura 1**).

En python podemos implementar el optimizador ingenuo para el modelo de regresión lineal, fijando 1000 épocas y calculando la pérdida con *MSE* (**Figura 2**).

```
1 def Loss(y, y_predict):
2     e = y - y_predict
3     mse = e.dot(e).mean()
4     # mae = numpy.abs(e).mean()
```

```

5     return mse
6
7     beta_opt = numpy.random.normal(0, 1)
8     bias_opt = numpy.random.normal(0, 1)
9
10    y_predict_opt = precios_dolar * beta_opt + bias_opt
11    loss_opt = Loss(y, y_predict_opt)
12
13    print(beta_opt, bias_opt, loss_opt)
14
15    print("-" * 80)
16
17    for epoca in range(1_000):
18        beta_test = numpy.random.normal(beta_opt, 1000. / (
19            epoca + 1))
20        bias_test = numpy.random.normal(bias_opt, 1000. / (
21            epoca + 1))
22
23        y_predict_opt = precios_dolar * beta_opt + bias_opt
24        y_predict_test = precios_dolar * beta_test + bias_test
25
26        loss_opt = Loss(y, y_predict_opt)
27        loss_test = Loss(y, y_predict_test)
28
29        if loss_test < loss_opt:
30            beta_opt = beta_test
31            bias_opt = bias_test
32
33        y_predict_opt = precios_dolar * beta_opt + bias_opt
34        loss_opt = Loss(y, y_predict_opt)
35
36    print(beta_opt, bias_opt, loss_opt)

```

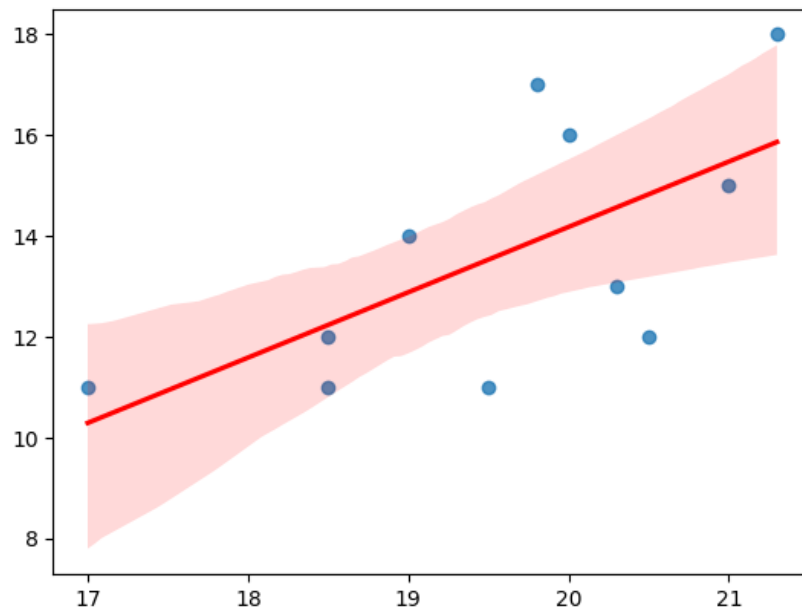


Figura 1: Visualización del modelo de regresión lineal y sus bandas

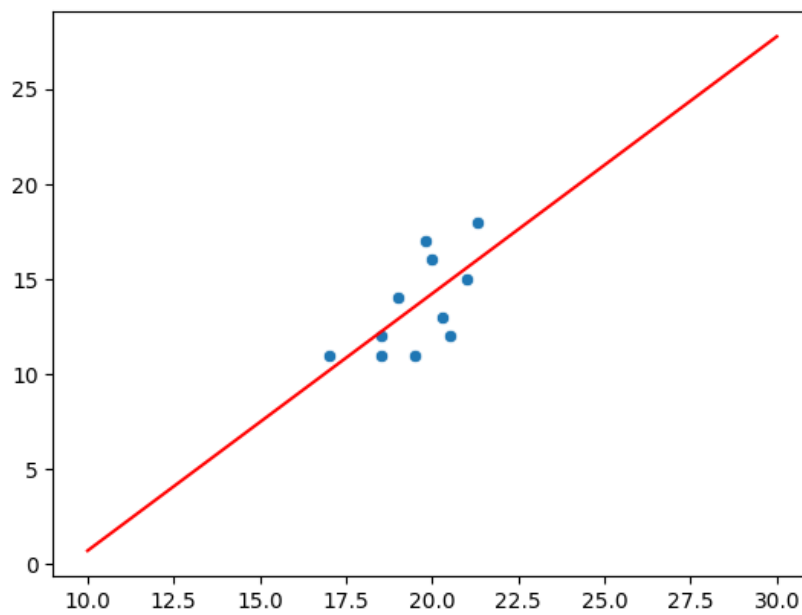


Figura 2: Resultados del modelo de regresión lineal optimizado

7. La curva logística como función de activación y el perceptrón

Cuando el espacio de predicción (la respuesta observada) está acotada entre cero y uno (Espacio binario o probabilístico), podemos diseñar una curva que en su límite negativo se vuelva cero y en su límite positivo se converja a uno, a esta curva la llamaremos la curva logística.

$$y \approx \hat{y} = \phi(x, \beta) = \frac{e^{x \cdot \beta_1 + \beta_0}}{1 + e^{x \cdot \beta_1 + \beta_0}} \quad (11)$$

A diferencia de la regresión lineal, ahora la función objetivo toma el predictor lineal para exponenciarlo y dividirlo al mismo tiempo por la exponencial más uno. A esta curva logística también se le conoce como el perceptrón con función de activación sigmoide o sigmoidal.

En términos simples, podemos pensar al predictor lineal como una percepción capturada (y balanceada por pesos), de los datos observados:

$$z = x \cdot \beta + \beta_0 \quad (12)$$

A este valor z lo conoceremos como el valor de percepción o activación lineal alcanzada, cuando ese valor es adaptado por una función de activación, entonces se genera una respuesta activada de la percepción:

$$\hat{h} = \phi(z) = \phi(x \cdot \beta + \beta_0) \quad (13)$$

Estas activaciones intermedias $\hat{h}$ son las respuestas del perceptrón, y cuando se combinan surgen las redes neuronales.

Los perceptrones se utilizan para ajustar datos en espacios probabilísticos o binarios, por ejemplo, clasificación binaria donde hay que determinar si un valor alcanzará el 1 o alcanzará el 0, como en la supervivencia del Titanic, para determinar si un pasajero sobrevive (respuesta 1) o muere (respuesta 0), de forma similar para determinar si alguien tiene diabetes, o la probabilidad de ser hospitalizado si tiene COVID más otros factores metabólicos o respiratorios.

Veamos un ejemplo que permite diferenciar entre una regresión lineal y una curva logística cuando los datos no son lineales e intentan predecir una respuesta probabilística (**Figura 3** y **Figura 4**).

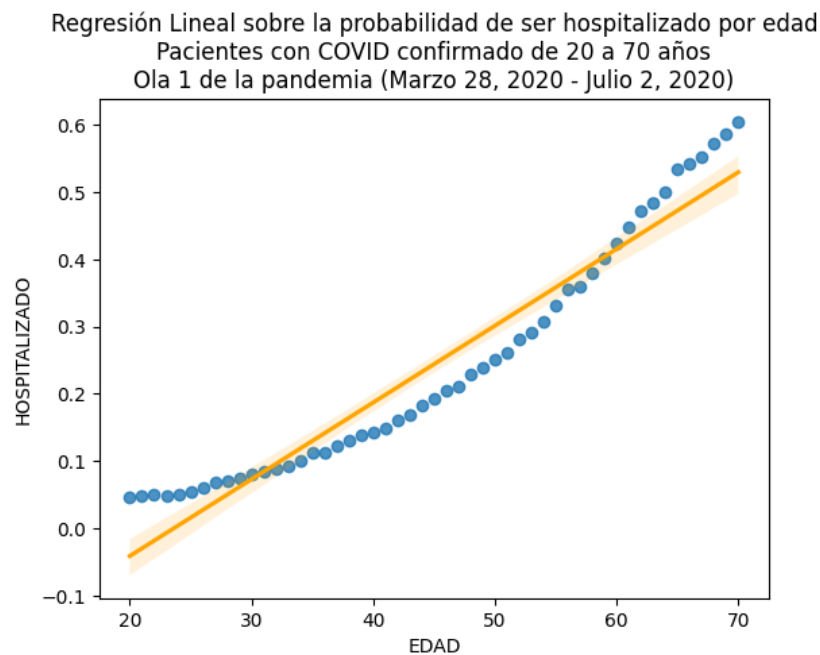


Figura 3: Modelo de regresión lineal sobre una respuesta probabilística

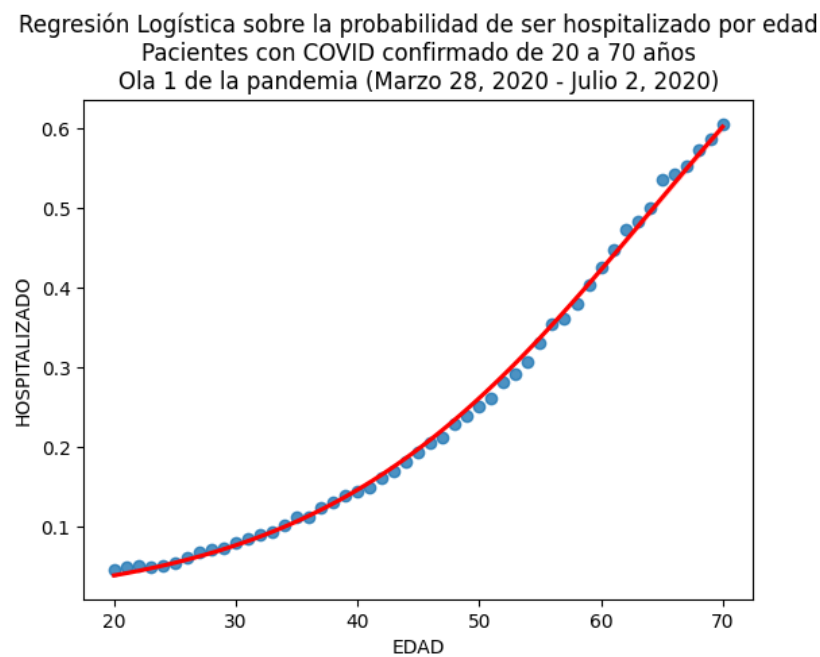


Figura 4: Modelo de la curva logística sobre una respuesta probabilística

8. Tarea 1

En un evento de anime ingresan personas de distintas edades, pero generalmente tienen entre 16 y 30 años. Nuestro objetivo es analizar las diferencias de edades entre las personas que van ingresando, para intentar predecir si entran grupos de personas de edades similares o hay un comportamiento variable.

Resuelve los siguientes puntos:

- Diseña una lista, arreglo o serie de edades con valores que creas realistas.
- Calcula la diferencia de las edades
- Grafica las edades y las diferencias
- Crea una lista, arreglo o serie de edades menores a 20 años que crees que asistiría la convención y otra de mayores de 20 años.
- Mezcla ambas listas en una sola y desordenala.
- Repite el cálculo de diferencias y las gráficas.

8.1. Preguntas de análisis

Contesta las siguientes preguntas de análisis:

1. ¿Cómo podrías determinar que un grupo de personas de edades similares acaba de entrar?
2. ¿Cuál sería la venta (número de personas) adecuado para observar?
3. ¿Qué representaría el promedio de una venta de edades?
4. ¿Tiene sentido el último cálculo de desordenar la lista o se perdería alguna información?
5. ¿Qué sería lo más conveniente para mezclar ambas listas o cuál procedimiento será más adecuado?

Nota: La tarea se deberá entregar en formato PDF con las fotos o captura de pantalla de los resultados, no es necesario mostrar el código. No olvides poner tu nombre y las repuestas a las preguntas de análisis.

9. Conclusiones

En esta sesión hemos podido familiarizarnos con python a través de ejemplos y problemas comunes en el cómputo científico y el análisis de datos. Más aún hemos explorado algunos problemas de interés relacionados con la inteligencia artificial, que nos permitirá en próximas clases afinar modelos más avanzados y poder resolver problemas más profundos.

Nota: Estas notas serán actualizado con lo expuesto en clase como complemento de estudio.

Bibliografía

- Géron, A. (2023). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (3.^a ed.). O'Reilly Media.
- Chollet, F. (2021). Deep Learning with Python (2.^a ed.). Manning Publications.
- Raschka, S., & Mirjalili, V. (2022). Python Machine Learning: Machine Learning and Deep Learning with Python, scikit-learn, and TensorFlow (3.^a ed.). Packt Publishing.
- Müller, A. C., & Guido, S. (2016). Introduction to Machine Learning with Python: A Guide for Data Scientists. O'Reilly Media.
- Grus, J. (2019). Data Science from Scratch: First Principles with Python (2.^a ed.). O'Reilly Media.
- Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4.^a ed.). Pearson.